

following the underscore prefix convention for private access, you make debugging easier. The reason is that you know the cause of the bug will be in the code in the class instead of anywhere in the entire program.

Note that unlike Java and other languages, Python has no need for public getter and setter methods for private attributes. Instead Python uses properties, as explained in Chapter 17.

The `type()` Function and `__qualname__` Attribute

Passing an object to the built-in `type()` function tells us the object's data type through its return value. The objects returned from the `type()` function are type objects, also called *class* objects. Recall that the terms *type*, *data type*, and *class* all have the same meaning in Python. To see what the `type()` function returns for various values, enter the following into the interactive shell:

```
>>> type(42) # The object 42 has a type of int.
<class 'int'>
>>> int # int is a type object for the integer data type.
<class 'int'>
>>> type(42) == int # Type check 42 to see if it is an integer.
True
>>> type('Hello') == int # Type check 'Hello' against int.
False
>>> import wizcoin
>>> type(42) == wizcoin.WizCoin # Type check 42 against WizCoin.
False
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> type(purse) == wizcoin.WizCoin # Type check purse against WizCoin.
True
```

Note that `int` is a type object and is the same kind of object as what `type(42)` returns, but it can also be called as the `int()` constructor function: the `int('42')` function doesn't convert the `'42'` string argument; instead, it returns an integer object based on the argument.

Say you need to log some information about the variables in your program to help you debug them later. You can only write strings to a